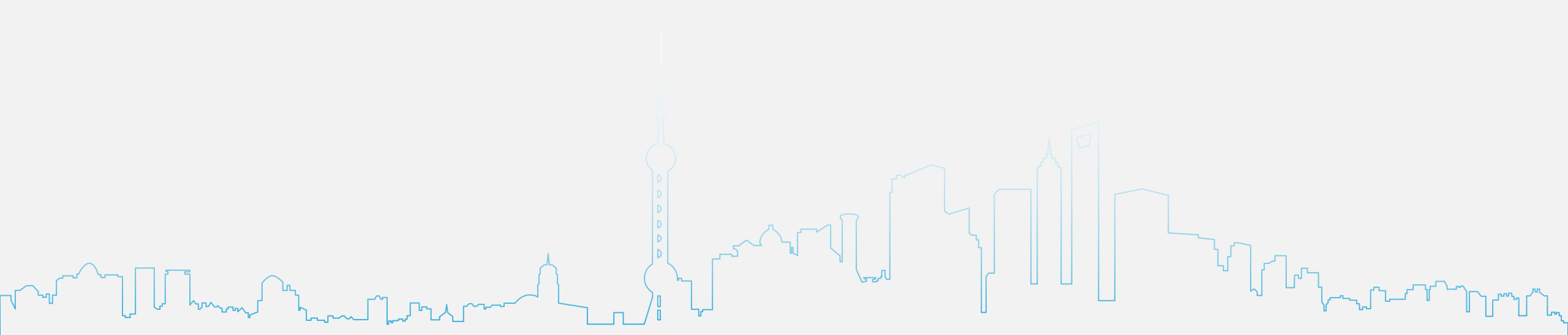


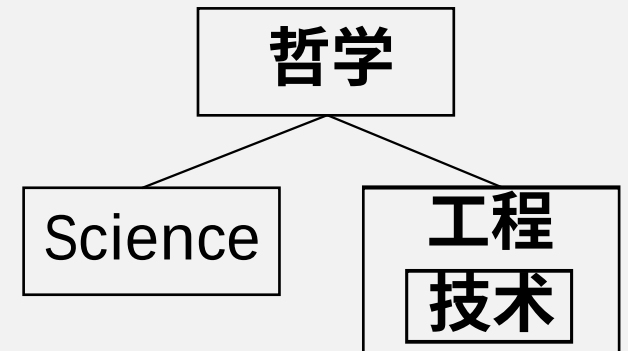


第三章 1 有符号整数和有理数的二进制表示

(Binary Representation of Signed Integer and Signed Rational Numbers)

秦磊华 汪承宁

Science是（增加或深度）**理解**世界（的空间和时间维度的**普适性规律**），是由大到小的过程（分析）。Engineering是**改变**世界（由人来做），是由小到大的过程（综合）。
技术是工程中很小的、巧妙的点。





3.4 数值数据表示

1. 有符号数 (signed numbers)

- ◆ 二进制数
- ◆ 符号数值化的方法?

└ 用0、1表示 { 表示简单
 处理简单

Note：数据类型 (data type) 属于指令集架构 (ISA, 软硬件界面) 的附属属性。



3.4 数值数据表示

2. 定点数 (fixed-point numbers)

小数点的位置**固定**

$$\begin{array}{ccccccc} X_n & X_{n-1} & X_{n-2} & \dots & X_1 & \cdot & X_0 \\ & & & & & \color{red}{\blacktriangle} & \\ X_n & X_{n-1} & \cdot & X_{n-2} & \dots & X_1 & X_0 \\ & & \color{red}{\blacktriangle} & & & & \end{array}$$

$$\left. \vphantom{\begin{array}{ccccccc} X_n & X_{n-1} & X_{n-2} & \dots & X_1 & \cdot & X_0 \\ X_n & X_{n-1} & \cdot & X_{n-2} & \dots & X_1 & X_0 \end{array}} \right\} X_i \in \{ 0, 1 \}$$

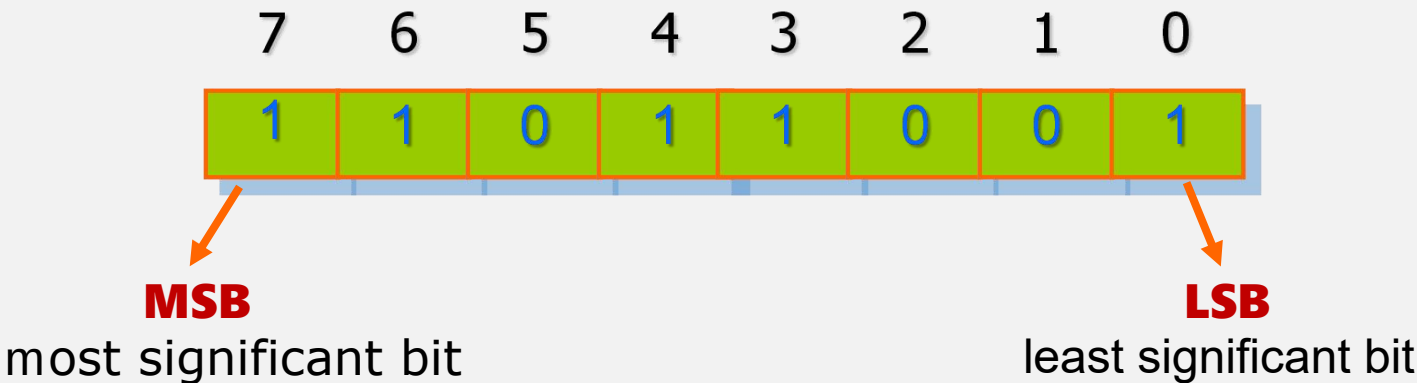
◆ 表示n+1位二进制**定点数**(含符号位)需要多少位二进制?

常见格式：

Q1.15

Q1.31

U8.8



Ref: Harris & Harris, DDCA, Chapter 5

3. 原码 (sign + magnitude)

$X_n X_{n-1} X_{n-2} \dots X_1 X_0$ ($n+1$ 位, 数值位为 n 位)

- 1) 符号位: 0表示正数, 1表示负数 (关于0.5对称) ;
- 2) 数值部分与绝对值相同
- 3) 0存在+0和-0两种编码, 但值一样。

整数 $[X]_{\text{原}} = \begin{cases} X & 0 \leq X \leq 2^n - 1 \\ 2^n - X & -(2^n - 1) \leq X \leq 0, \text{ 符号参与运算} \end{cases}$

$x = + 1011001 \quad \longrightarrow \quad [x]_{\text{原}} = 01011001$

$x = - 1011001 \quad \longrightarrow \quad [x]_{\text{原}} = 11011001$

该表示方法也称为符号加幅度 (sign + magnitude)



3.4 数值数据表示

3. 原码

$$X_n.X_{n-1}X_{n-2}...X_1X_0$$

1) 表示方法： 符号位: 0表示正数， 1表示负数， 数值位与真值相同

纯小数 $[X]_{\text{原}} = \begin{cases} X & 0 \leq X < 1 \\ 1 - X & -1 < X \leq 0 \end{cases}$ 符号参与运算

$x = + 0.1011001 \longrightarrow [x]_{\text{原}} = 0.1011001$

$x = - 0.1011001 \longrightarrow [x]_{\text{原}} = 1.1011001$

对于纯小数，整数部分的一个0不用存储（信息不会丢失）。



3.4 数值数据表示

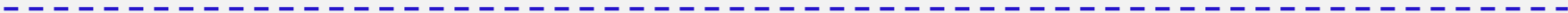
3. 原码 2) 原码数据表示范围

共 $n+1$ 位： $X_n X_{n-1} X_{n-2} \dots X_1 X_0$ 求绝对值简单，在数轴上两边对称。

整数

$111\dots1 \sim 01111\dots1 \longrightarrow [-(2^n-1), 2^n-1]$

$11111111 \sim 01111111 \longrightarrow [-127, 127]$



纯小数

$1.1111\dots1 \sim 0.1111\dots1 \longrightarrow [-(1-2^{-n}), 1-2^{-n}]$

$1.1111111 \sim 0.1111111 \longrightarrow [-(1-2^{-7}), 1-2^{-7}]$

3.4 数值数据表示

3. 原码

3) 原码数据表示的特点

整数 $[X]_{\text{原}} = \begin{cases} X & 0 \leq X < 2^n \\ 2^n - X & -2^n < X \leq 0 \end{cases}$

纯小数 $[X]_{\text{原}} = \begin{cases} X & 0 \leq X < 1 \\ 1 - X & -1 < X \leq 0 \end{cases}$

◆ 0的表示有两种: 0000...0, 1000...0

◆ 符号位无法与数字位相同处理

◆ 加减运算较为复杂, 不能直接运算

◆ 数据表示区间对称

$$[-(1-2^{-n}), 1-2^{-n}]$$

$$[-(2^n-1), 2^n-1]$$



3.4 数值数据表示

4. 反码

$X_nX_{n-1}X_{n-2}\dots X_1X_0$ (共 $n+1$ 位, 数值位为 n 位)

0表示正数, 数值位与真值或原码相同

利用整数重要性质： 1表示负数, 数值位与真值或原码相反 (每一位都对 $2-1=1$ 求补)

$A_n + \overline{A_n} = 2^n - 1$
bar表示按位求对1的补

十进制 $\overline{26} = 73$
每一位都对 $10-1=9$ 求补

$[X]_{反} = \begin{cases} X, & 0 \leq X \leq 2^n - 1 \\ (2^{n+1} - 1) - (-X), & -(2^n - 1) \leq X \leq 0 \end{cases}$

$x = + 1011001$	----->	$[x]_{反} = 0\ 1011001$
$x = - 1011001$	----->	$[x]_{反} = 1\ 0100110$

如何实现?
↓
非门阵列



3.4 数值数据表示

4. 反码

共n+1位 $X_n.X_{n-1}X_{n-2}...X_1X_0$

- 1) 表示方法： 0表示正数，数值位与真值或原码相同
1表示负数，数值位与真值或原码相反

纯小数 $[X]_{反} = \begin{cases} X & 0 \leq X < 1 \\ 2 - 2^{-n} - (-X) & 1 < X \leq 0 \end{cases}$

利用纯小数重要性质：

$$A_n + \overline{A}_n = 1 + [1 - 2^{-(n-1)}]$$

$x = + 0.1011001 \longrightarrow [x]_{反} = 0.1011001$

$x = - 0.1011001 \longrightarrow [x]_{反} = 1.0100110$

最高位的整数0不存储，符号位看作最低位的整数位



3.4 数值数据表示

4. 反码

2) 反码数据表示范围

$$X_n X_{n-1} X_{n-2} \dots X_1 X_0$$

整数

反码: $10000\dots0 \sim 01111\dots1 \longrightarrow [-(2^n-1), 2^n-1]$
(真值: $-1111\dots1 \sim +1111\dots1$)

$10000000 \sim 01111111 \longrightarrow [-127, 127]$
(真值: $-1111\dots1 \sim +1111\dots1$)

纯小数

求负数的绝对值
 $1.0000\dots0 \sim 0.1111\dots1 \longrightarrow [-(1-2^{-n}), 1-2^{-n}]$

$1.00000000 \sim 0.11111111 \longrightarrow [-(1-2^{-7}), 1-2^{-7}]$

注意最高位的整数0省去不存储



3.4 数值数据表示

4. 反码

3) 反码数据表示的特点

0的表示有2种： $0000...0 (+0)$, $1111...1 (-0)$
 $+0 = -0$

◆ 运算相对复杂：加运算需将进位位与最低位相加

$$\begin{array}{r} 1101 \quad (-2) \\ + \quad 1010 \quad (-5) \\ \hline C=1 \quad 0111 \\ \quad \quad 1 \\ \hline 1000 \quad (-7) \end{array}$$

◆ 符号位参与运算

◆ 数据表示区间关于原点对称

$$\begin{aligned} &[-(1-2^{-n}), 1-2^{-n}] \\ &[-(2^n-1), 2^n-1] \end{aligned}$$

$$[X]_{\text{反}} = \begin{cases} X & 0 \leq X < 2^n \\ 2^{n+1} - 1 - (-X) & -2^n < X \leq 0 \end{cases}$$

对(2的整数次幂-1)求补

$$[X]_{\text{反}} = \begin{cases} X & 0 \leq X < 1 \\ 2 - 2^{-n} + X & -1 < X \leq 0 \end{cases}$$



3.4 数值数据表示

5. 补码 (two's complement)

$$w = -a_{N-1}2^{N-1} + \sum_{i=0}^{N-2} a_i 2^i$$

1)模 (modulo) 的概念：符号位进位位的权值

- ◆ 字长不同，模(符号位进位位的权值)不同
- ◆ 字长为n，则权值为 2^n

$$1010 = -(1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) = 1 \times -8 + 0 + 1 \times 2 + 0 = -6.$$

Bits:	1	0	1	0
Decimal bit value:	-8	4	2	1
Binary calculation:	$-(1 \times 2^3)$	(0×2^2)	(1×2^1)	(0×2^0)
Decimal calculation:	$-(1 \times 8)$	0	1×2	0

$10...000_2 = -2^{N-1}$
奇怪数 (weird number)：按位求对1的补再加1得到自身，没有对应正数



3.4 数值数据表示

5. 补码

整数： $X_nX_{n-1}X_{n-2}...X_1X_0$ (共 $n+1$ 位)

重要性质：

$\overline{A_n+1}+1 = A_n$

bar表示按位求反

0表示正数，数值位与绝对值相同
1表示负数，数值位在反码最低位加1

$[X]_{补} = \begin{cases} X & 0 \leq X < 2^n \\ 2^{n+1} - (-X) & -2^n \leq X < 0 \end{cases} \pmod{2^{n+1}}$

对2的整数次幂求补，故名补码

modulo属于带余除法
(保持整数域)，注意
余数 (remainder) 的
非负性！
例如 $-5/16 = -1$ 余 11

$x = + 1011001 \longrightarrow [x]_{补} = 01011001$
 $x = - 1011001 \longrightarrow [x]_{补} = 10100111$



3.4 数值数据表示

5. 补码

纯小数： $X_n.X_{n-1}X_{n-2}...X_1X_0$

0表示正数，数值位与真值或原码相同

1表示负数，数值位在反码最低位加1

$$[X]_{\text{补}} = \begin{cases} X & 0 \leq X < 1 \\ 2 + X & -1 \leq X < 0 \end{cases} \quad \text{mod } 2^{n+1}$$

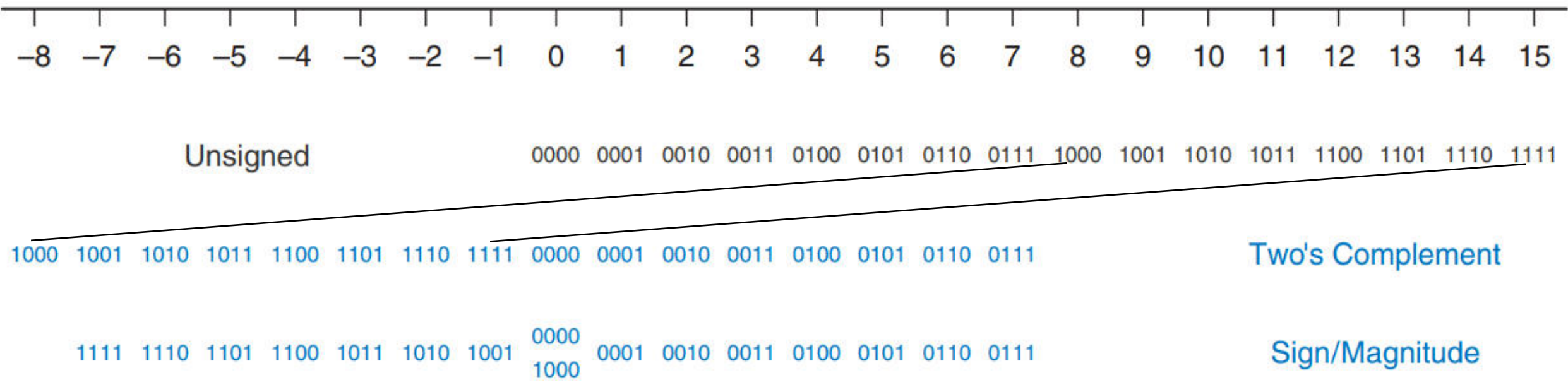
$x = + 0.1011001 \quad \text{-----} \rightarrow \quad [x]_{\text{补}} = 0.1011001$

$x = - 0.1011001 \quad \text{-----} \rightarrow \quad [x]_{\text{补}} = 1.0100111$

3.4 数值数据表示

5. 补码 3) 补码数值表示范围

整数	10000...0	~	01111...1	----->	$[-2^n, 2^n-1]$
	10000000	~	01111111	----->	$[-128, 127]$
纯小数	1.0000...0	~	0.1111...1	----->	$[-1, 1-2^{-n}]$
	1.0000000	~	0.1111111	----->	$[-1, 1-2^{-7}]$



5. 补码

4) 补码的数据表示特点

计算-7的4位二进制补码：
 偏移量 $\text{bias}=2^4=16$, $-7 + 16 = 9$, 9的无符号表示为1001, 即为所求

$$[X]_{\text{补}} = \begin{cases} X & 0 \leq X < 2^n \\ 2^{n+1} + X & -2^n \leq X < 0 \end{cases}$$

特征：负数的表示采用偏移码，
 给一个偏移量

$$[X]_{\text{补}} = \begin{cases} X & 0 \leq X < 1 \\ 2 + X & -1 \leq X < 0 \end{cases}$$

- ◆ 0的表示唯一：0000...0
 - ◆ 运算简单：符号位和数字位参与相同的运算
 (可写成统一的幂次多项式)
 - ◆ 负数和正数表示区间不对称，大于同位宽的原码和反码
- 纯小数 $[-1, 1-2^{-n}]$

整数 $[-2^n, 2^n-1]$



3.4 数值数据表示

5. 补码

6) 变形补码

$$[X]_{\text{补}} = \begin{cases} X & 0 \leq X < 2^n \\ 2^{n+2} + X & -2^n \leq X < 0 \end{cases} \quad \text{mod } 2^{n+2}$$

$$[X]_{\text{补}} = \begin{cases} X & 0 \leq X < 1 \\ 4 + X & -1 \leq X < 0 \end{cases} \quad \text{mod } 2^2$$

$x = + 1011001$	----->	$[x]_{\text{补}} = 00 \ 1011001$
$x = - 1011001$	----->	$[x]_{\text{补}} = 11 \ 0100111$



3.4 数值数据表示

5. 补码 7) 补码的符号扩展 (sign extension)

$[X]_{补} = 0101010111000011$ $[Y]_{补} = 10011100$ 求 $[X]_{补} + [Y]_{补} = ?$

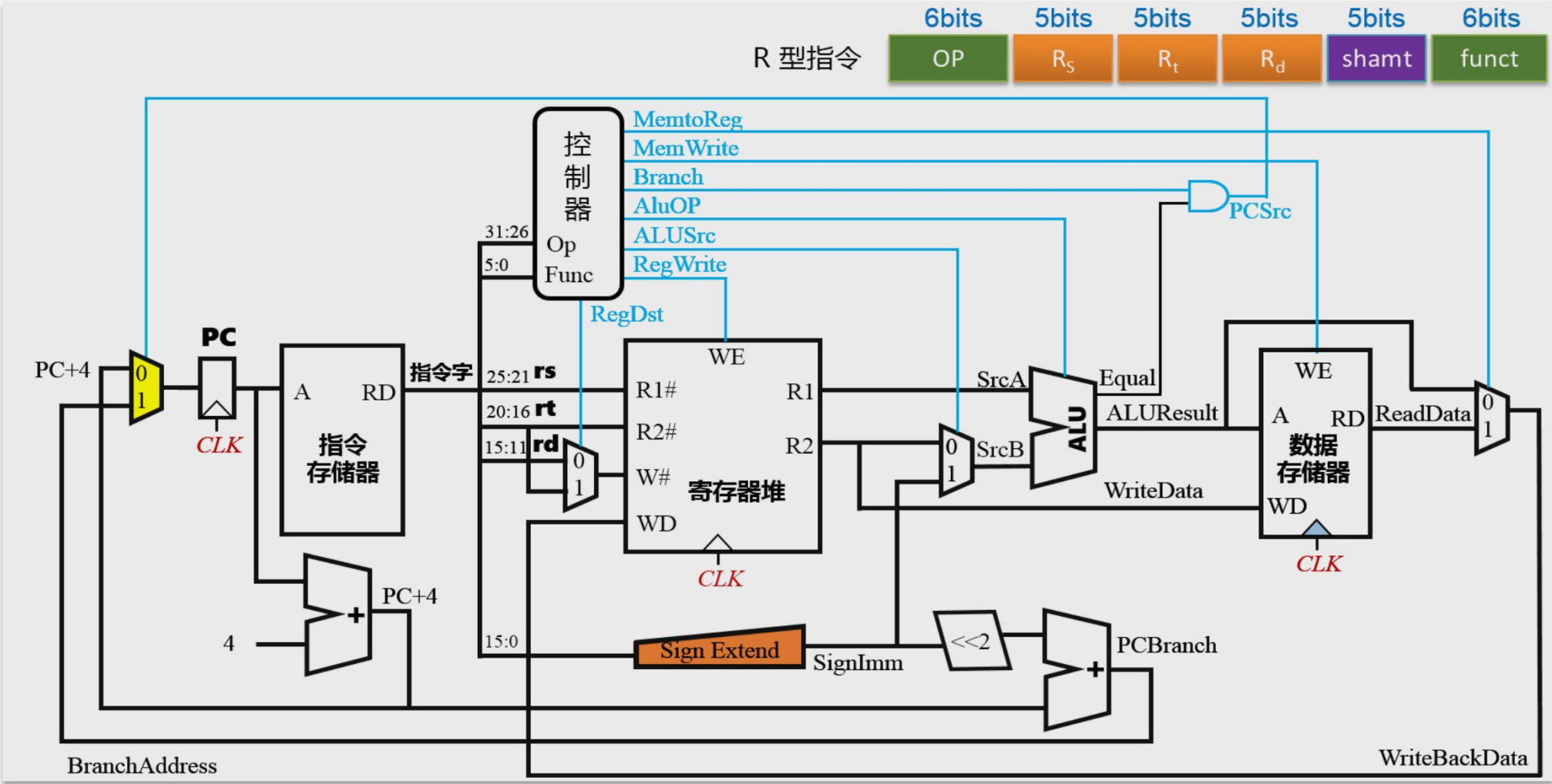
0	1	0	1	0	1	0	1	1	1	0	0	0	0	1	1																							
															+																1	0	0	1	1	1	0	0
????????????????????																																						

0	1	0	1	0	1	0	1	1	1	0	0	0	0	1	1																													
															+	1															1	1	1	1	1	1	1	0	0	1	1	1	0	0
0101010101011111																																												

根据补码的多项式定义，利用 $-1 \cdot 2^{n-1} = -1 \cdot 2^n + 1 \cdot 2^{n-1}$ ，再不断向左递推，可得。

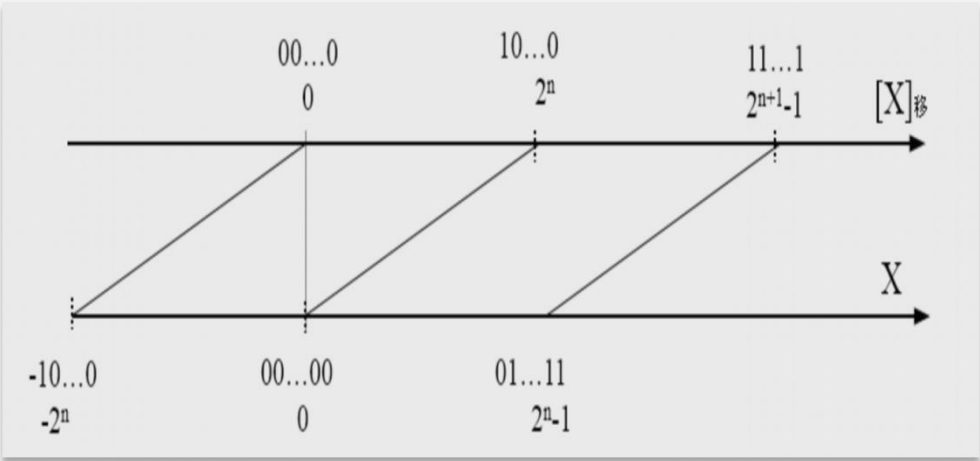
符号扩展，原值不变！
不同位数的整数补码相加时，位数少的向位数多的数进行符号扩展。

5. 补码 符号扩展模块在微架构中的应用



6. 移码(增码) 偏移码 $X_nX_{n-1}X_{n-2}\dots X_1X_0$

给一个偏移量bias, 例如 $[x]_{\text{移}} = 2^n + x \quad -2^n \leq x < 2^n$



保持数据原有大小顺序, 便于比较

◆与补码的符号位相反, 数据位相同

$$X = +10101 \quad [X]_{\text{移}} = 2^5 + 10101 = 110101$$

$$X = -10101 \quad [X]_{\text{移}} = 2^5 - 10101 = 001011$$

◆用于表示浮点数的阶码, bias = 127

Format	Total Bits	Sign Bits	Exponent Bits	Fraction Bits	Bias
Single	32	1	8	23	127
Double	64	1	11	52	1023
Quad	128	1	15	112	16383

7. 定点数据表示小结

将十进制值X(-127, -1, 0, 1, 127) 用四种二进制存储格式表示

X	真值	[X] _原	[X] _反	[X] _补	[X] _{移, bias=128}
-127	-01111111	11111111	10000000	10000001	00000001
-1	-00000001	10000001	11111110	11111111	01111111
0	00000000	10000000	11111111	00000000	10000000
		00000000	00000000		
1	+00000001	00000001	00000001	00000001	10000001
127	+11111111	01111111	01111111	01111111	11111111



3.4 数值数据表示

n+1位**定点**二进制数存储格式的表示范围

	整数	纯小数
原码/反码	$[-(2^n-1), 2^n-1]$	$[-(1-2^{-n}), 1-2^{-n}]$
补码	$[-2^n, 2^n-1]$	$[-1, 1-2^{-n}]$
移码	$[-2^n, 2^n-1]$	纯小数无移码



3.4 数值数据表示

8. 高级编程语言中的定点数

1) 无符号整数

unsigned char

unsigned short

unsigned int

unsigned long long

一般用于地址运算，编号表示

2) 有符号整数

◆ char short int long long

◆ 采用补码表示

3) 无符号数与有符号数的取值范围(以8位补码为例)

-128 ~ 127 VS 0 ~ 255

3.4 数值数据表示

32位ISA上的程序

```
int main(void)
{
    int x=-1;
    unsigned int u = 2147483648;
    printf ("x = %u = %X = %d\n",x,x,x);
    printf ("u = %u = %X = %d\n",u,u,u);
    return 0;
}
```

真值赋值

真值输出

打印结果：x = 4294967295 = FFFFFFFF = -1

u = 2147483648 = 80000000 = -2147483648

9. 汇编语言中的数据类型

数据类型（data type）属于ISA（软硬件界面）的一部分。
 取决于指令的**操作码（opcode）**。

ISA	无符号运算	有符号运算	浮点运算
X86	ADD/SUB 加减		F ADD/ F SUB 加减
	MUL/DIV 乘除	I MUL/ I DIV 乘除	F MUL/ F DIV 乘除
MIPS32	ADD U /SUB U 加减	ADD/SUB 加减	ADD. S ADD. D / SUB. S SUB. D 加减
	MULT U /DIV U 乘除	MULT/DIV 乘除	MUL. S MUL. D / DIV. S DIV. D 乘除
RISC-V32	ADD/SUB 加减		F ADD. S F ADD. D / F SUB. S F SUB. D 加减
	MULH U /DIV U 乘除	MULH/DIV 乘除	F MUL. S F MUL. D / F DIV. S F DIV. D 乘除

10. 浮点数据表示

$$\pm M \times B^E$$

1) 为什么要引入浮动的小数点 (floating-point) ，浮点？

◆ 表示太阳质量 1.989×10^{30} kg、电子质量 9.109×10^{-28} 需要多少个十进制位？

◆ 科学记数法 (scientific notation) 用符号、**指数 (exponent)** 和**尾数 (mantissa)** 来表示；二进制浮点数用符号、偏移指数和尾数 (fraction) 来表示。

能表示更大尺度范围的数。

◆ 除了区间范围限制外，固定格式、有限位宽的二进制浮点数**仅可表示一部分有理数**（可写成 a/b 的形式， a 和 b 是整数，且 $a:b$ 不可再约简）。不能表示无理数。

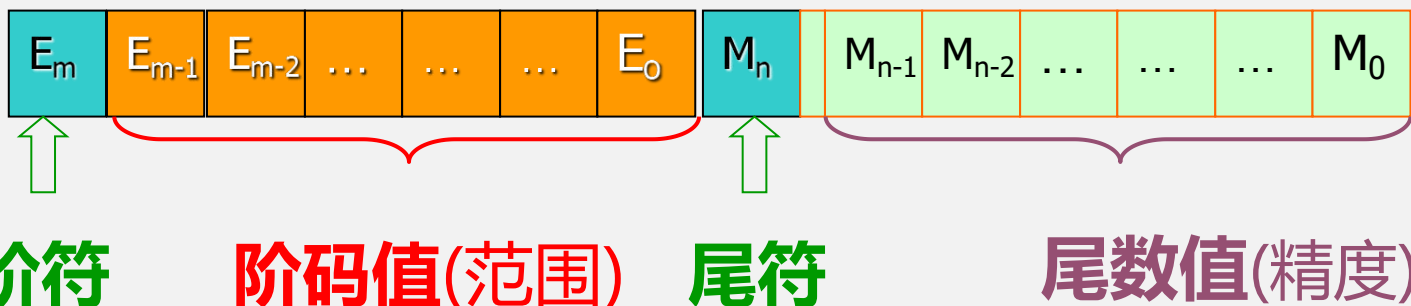
尽管有理数理论上可表示为分数，但浮点数的二进制存储特性会导致部分有理数无法精确表示。

当绝对值极大或极小时，浮点数的指数范围有限，可能因溢出或下溢导致误差。

10. 浮点数据表示

2) 计算机内浮点数的一般格式

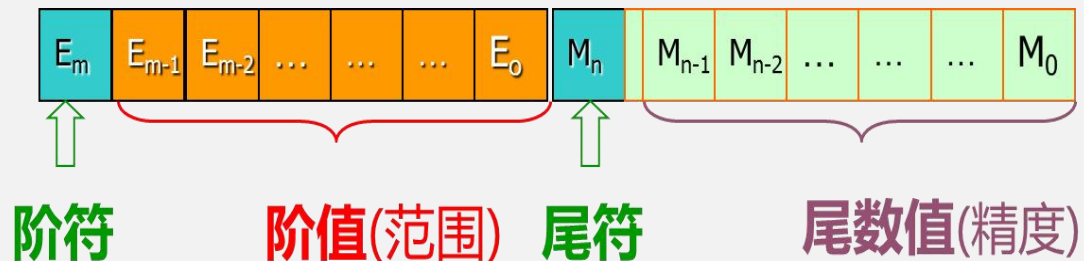
$$N = 2^{\pm e} \times (\pm m)$$



浮点数的引入主要是为了服务于计算数学、科学计算，尤其是用于指数动态范围 (dynamic range) 很大的时候，此时定点格式会带来较大的表示误差。

二进制浮点数无法表示大多数有理数和任何无理数（只能用其它的数近似）。对于一部分有理数，只能提供有限精度的近似值。无法准确表示无限循环小数和无限不循环小数。当有理数的分母可化为2的整幂次时可准确用二进制浮点数表示。

10. 浮点数据表示



- ◆ 字长确定的情况下，尾数位越长，则阶码越短，如何取舍？
- ◆ 早期各计算机公司不同型号的计算机，有着不同的浮点数表示；
- ◆ 数据交换、软件可移植性受到极大影响。

How to construct **reliable** systems using **unreliable** devices?

1) Quantization; 2) Adding Redundancy (hardware overhead) and encoding

3.4 数值数据表示

10. 浮点数据表示

4) IEEE 754标准

上世纪70年代后期开始，1985年完成浮点数标准IEEE 754制定，也是现阶段的标准。



www.cs.berkeley.edu/~wkahan/ieee754status/754story.html

UC Berkeley mathematics professor William Kahan

Goldberg D. What every computer scientist should know about floating-point arithmetic[J]. ACM computing surveys, 1991, 23 (1): 5-48.

10. 浮点数据表示

4) IEEE 754格式



- ◆ 组成：阶码E，尾数M，符号位S.
- ◆ $N = (-1)^S \times 1.M \times 2^e$ (尾数最高有效位leading 1隐藏，不存储)
- ◆ 阶码E用移码表示，无符号，偏移量 $bias=127、1023$ ， $E=e + 127$ 或1023;
- ◆ 为什么要用移码：预留出全0和全1，便于表示特殊值。
 $e_{min} = -126, e_{max} = 127 (bias=127);$



3.4 数值数据表示

符号位	阶码	尾数	表示的数据
0/1	255	1xxxx	NaN Not a Number *
0/1	255	非零 0xxxx	sNaN Signaling NaN **
0	255	0	+ ∞
1	255	0	- ∞
0/1	1~254	M	$(-1)^S \times (1.M) \times 2^{(E-127)}$ (规格化)
0/1	0	M (非零)	$(-1)^S \times (0.M) \times 2^{(-126)}$ (非规格化)
0/1	0	0	+0/-0

NaN(Not a Number)有两类:QNaN(Quiet NAN)和SNAN(Singaling NAN)。前者尾数部分最高位为1，常表示**未定义的算术运算结果，如除0**。后者尾数最高位0；常用于**标记未赋初值的变量**，以此来捕获异常。

返回NaN的运算有如下三种

1)至少有一个操作数 (operand) 是NaN的运算

2)没有定义语义的运算

◆除法运算: $0/0$ 、 ∞/∞ 、 $\infty/-\infty$ 、 $-\infty/\infty$ 、 $-\infty/-\infty$

◆乘法运算: $0 \times \infty$ 、 $0 \times -\infty$

◆加法运算: $\infty + (-\infty)$ 、 $(-\infty) + \infty$

◆减法运算: $\infty - \infty$ 、 $(-\infty) - (-\infty)$

3)**产生复数 (complex number) 结果的实数 (real number) 运算 (越域)**。例如:

◆对负数进行开偶次方的运算

◆对负数进行对数运算

◆对正弦或余弦函数值域以外的数进行反正弦或反余弦运算



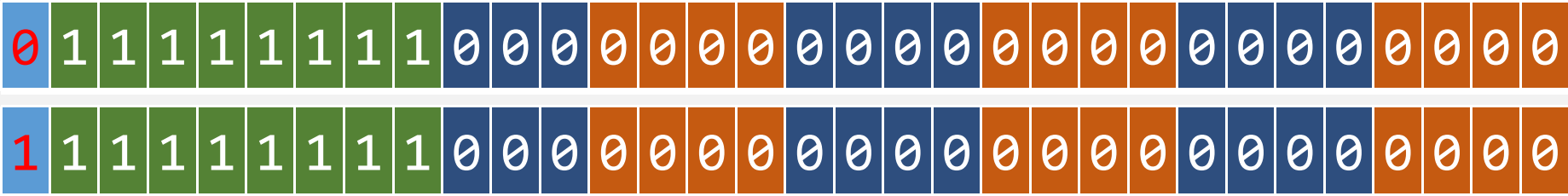
3.4 数值数据表示

```
int main(void)
{
    float a=1.0f, b=-1.0f;
    a /= 0.0f; b /= 0.0f;
    printf("a=%f b=%f", a, b);
    return 0;
}
```

符号位	阶码	尾数	表示的数据
0/1	255	1xxxx	NaN Not a Number *
0/1	255	非零 0xxxx	sNaN Signaling NaN **
0	255	0	+ ∞
1	255	0	- ∞
0/1	1~254	M	$(-1)^S \times (1.M) \times 2^{(E-127)}$ (规格化)
0/1	0	M (非零)	$(-1)^S \times (0.M) \times 2^{(-126)}$ (非规格化)
0/1	0	0	+0/-0

a=1.#INF00 b=-1.#INF00

无穷大 (infinity) 的缩写





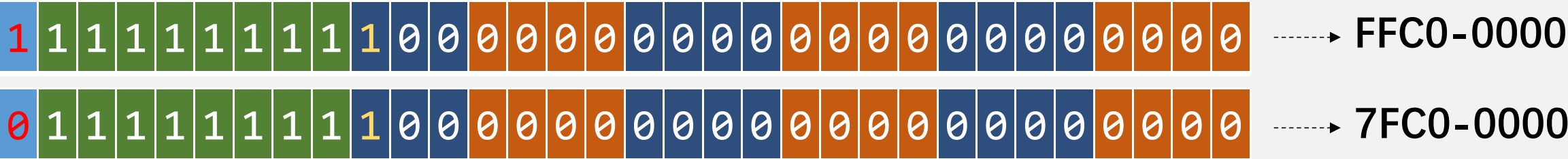
3.4 数值数据表示

```
int main(void)
{
    float a=0.0f, b;
    a /= 0.0f; b=-sqrt(-1.0f);
    printf ("a=%f b=%f",a,b);
    return 0;
}
```

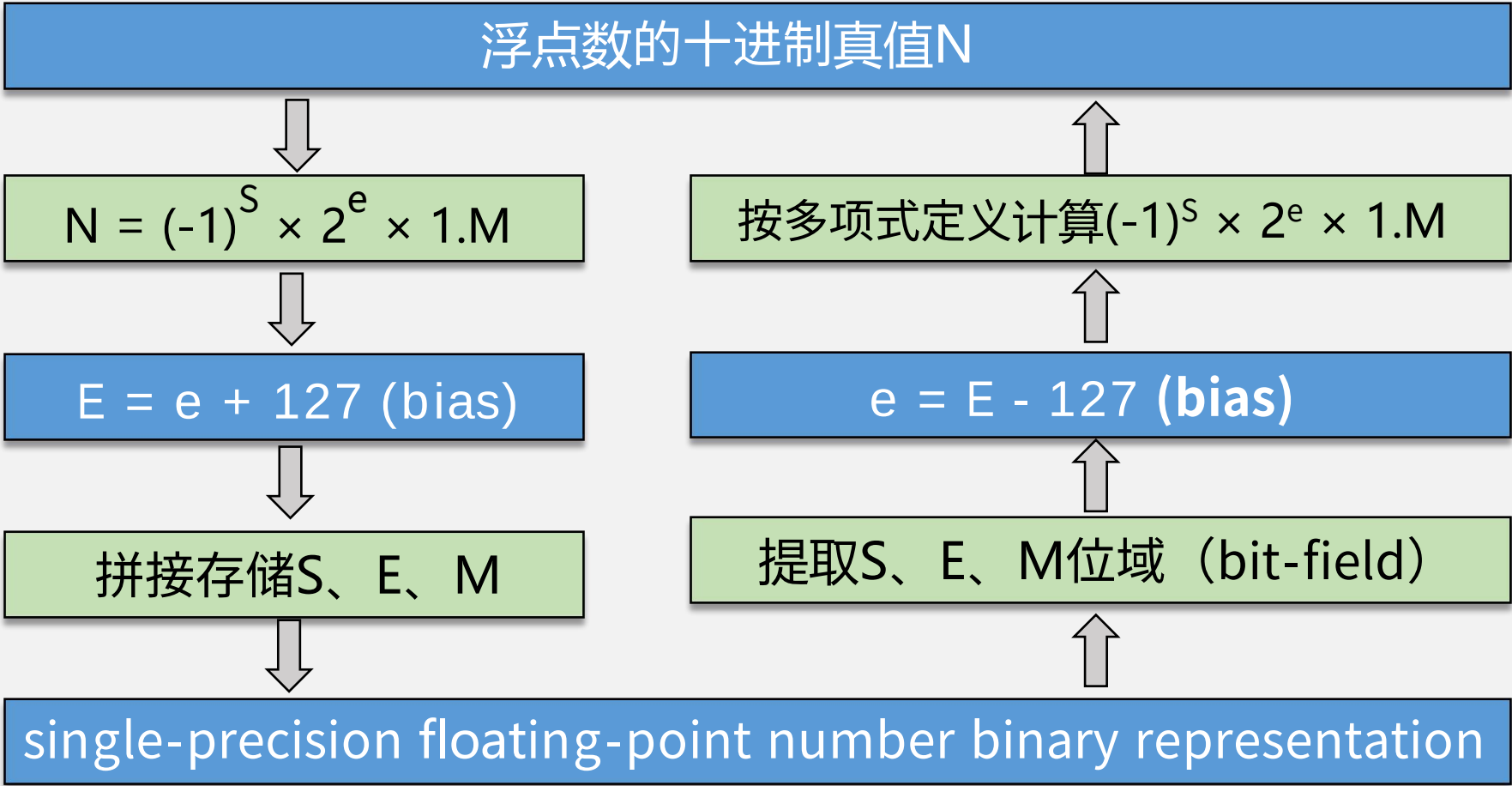
符号位	阶码	尾数	表示的数据
0/1	255	1xxxx	NaN Not a Number *
0/1	255	非零 0xxxx	sNaN Signaling NaN **
0	255	0	+ ∞
1	255	0	- ∞
0/1	1~254	M	$(-1)^S \times (1.M) \times 2^{(E-127)}$ (规格化)
0/1	0	M (非零)	$(-1)^S \times (0.M) \times 2^{(-126)}$ (非规格化)
0/1	0	0	+0/-0

a=1.#IND00 b=1.#QNAN0

IND: Implementation Not define, IND表示无限小，但不确定



10. 浮点数据表示 5) IEEE 754单精度浮点数格式与真值的转换



3.4 数值数据表示

10. 浮点数据表示

例 $3.3_{10} \rightarrow$ IEEE 754 single-precision FP

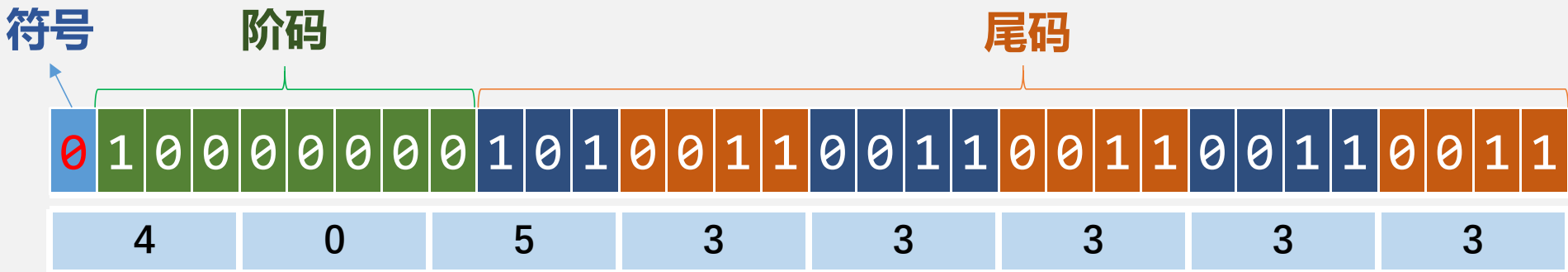
$$3.3_{10} = 11.01001100110011001100110011..._2$$

$$= 1.101001100110011001100110011 \times 2^1$$

$$S = 0, e = 1,$$

$$E = 01111111 + 1 = 10000000$$

$$M = 10100110011001100110011$$



3.4 数值数据表示

```
int main(void)
{
    double a,b,c;  int d;
    b=3.3;  c=1.1;
    a=b/c;
    d=(int)(b/c);
    printf("%f,%d\n",(float)a, d);
    if (3.0!=a)
        printf("3.0!=a\n");
    return 0;
}
```

3.000000,2
3.0!=a

根本原因：计算方法有问题。
b和c无法表示十进制有理数
3.3和1.1（相应的二进制是**无限循环小数**），只能用其它的数近似代替。与浮点表示无关，定点表示同样存在这样的问题。
解决方法：1) 在ISA层面扩展数据类型，采用有理数的约简分式（33/10, 11/10）存储分子和分母。2) 采用符号计算（代数）。

由于精确度损失问题，浮点运算不满足结合律

$$(2^{-126} + 10^{20}) - 10^{20} = ? \quad 2^{-126} + (10^{20} - 10^{20}) = ?$$